

Coming of Age: The Positive Legacy of FOSS GIS

Peter Löwe, GeoForschungsZentrum-Potsdam
ploewe@gfz-potsdam.de

Introduction

- Legacy is in the eye of the beholder
- A real legacy project: GRASS GIS
- Code Evolution and Legacy
- Legacy Code and Legacy Tools
- Arguing for Integrated Development Tools
- Conclusion

The meaning of Legacy [Wikipedia]

- **„Legacy code** is source code that relates to a no-longer supported or manufactured operating system or other computer technology“

„Forgotten Lore“: CP/M, BeOS etc.

- “Old application programs *are considered “legacy”* which continue to be used because the user does not want to replace or redesign them.”

„Never change working code“ – we don‘t why it works…

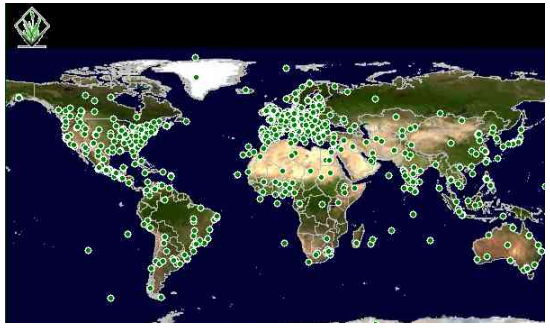
- “Legacy software often differs from its suggested alternative by actually working and scaling, making the term not pejorative at all.”

„It does the job“

FOSS GIS Legacy

Several FOSS GIS projects have been in existence for a very long time, giving them "legacy":

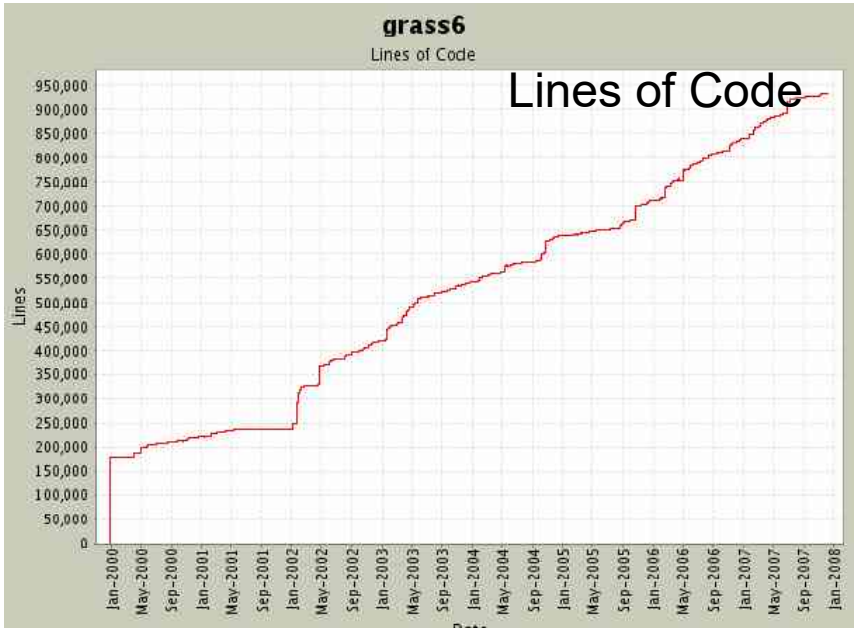
- UMN Mapserver
- GRASS GIS,
- both are community-controlled projects.



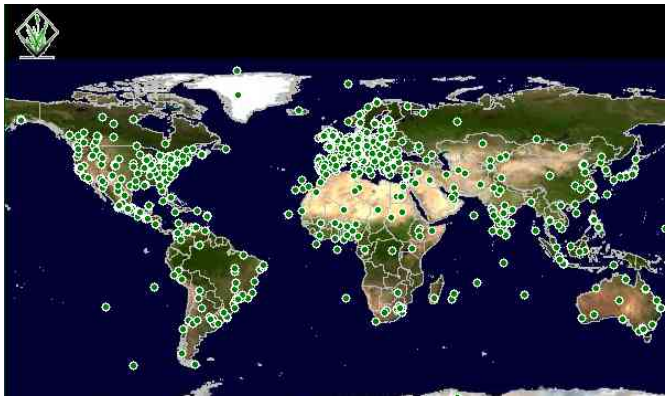
GRASS User Map

The „Code Legacy“ of GRASS: Alive and Kicking

- The GRASS GIS project has been evolving for more **than twenty years.**
- Development was started by the US Army (CERL). It was halted in the early 1990s: The sources were handed over to academia.
- Development was rekindled in the mid-1990s by researchers and students.
- The licensing under the General Public Licence (GPL) in 1999 resulted in a dramatic increase of development.
- GRASS GIS is now maintained by an international developer community.
- In 2008 it became a OSGEO project.



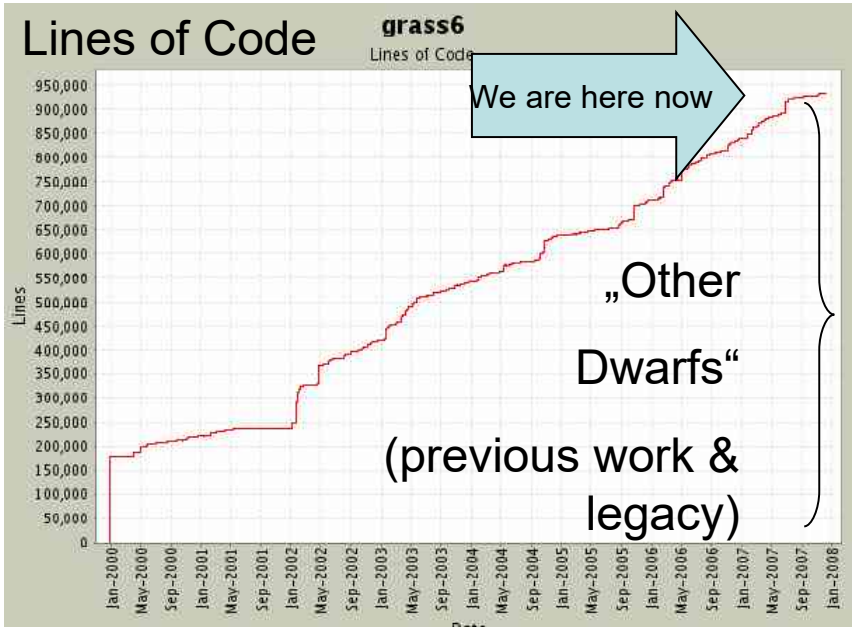
2000 2008



„Dwarfs standing on the shoulders of giants“

[nanos gigantum humeris insidentes]

- Analgoy for Free Open Source Code Development.
- „Current code is based on previous work“
- „dwarfs standing on lots of other dwarfs“



The metaphor was first recorded in the twelfth century and attributed to Bernard of Chartres, ***[Wikipedia]*** **(Later:** Sir I.Newton, R. Stallman)

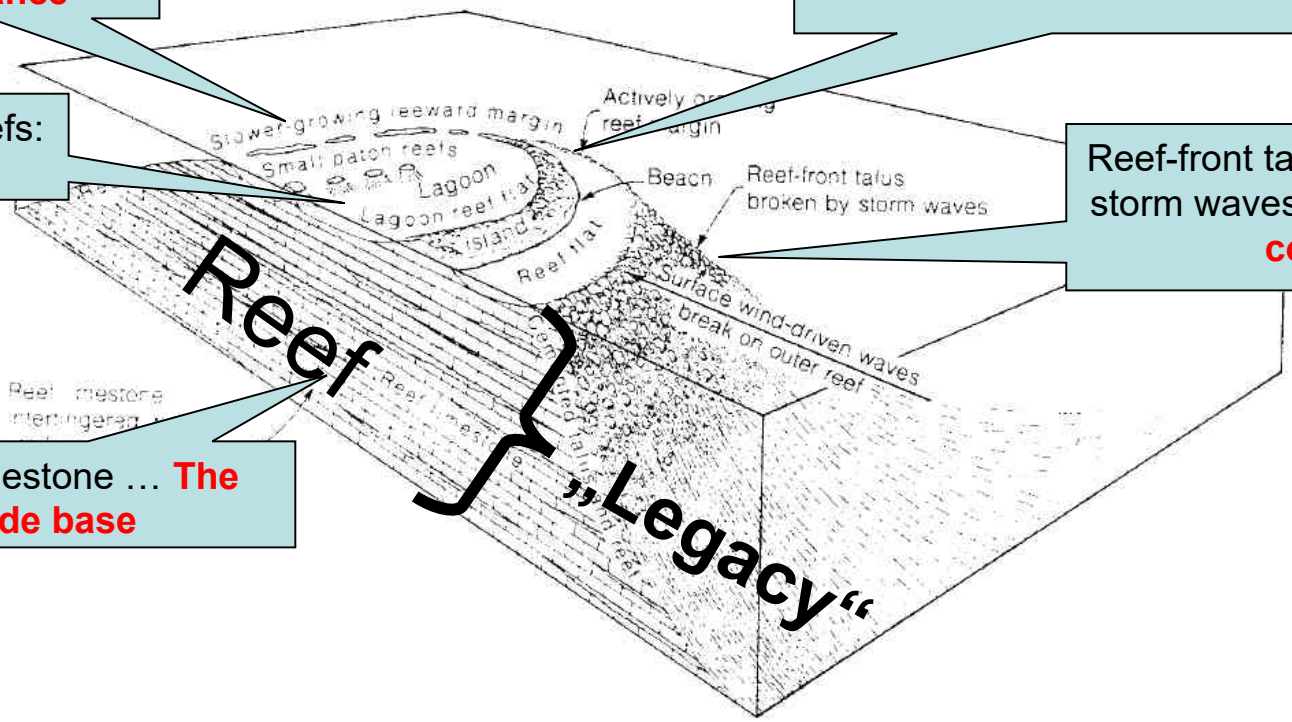
„Life is a Beach, Coding is a Reef“

Slow growing leeward margin: **Code maintenance**

Actively growing reef margin: **The cutting edge of development**

Small patch reefs: **Scripts, etc**

Reef-front talus, broken by storm waves: **Abandoned code**



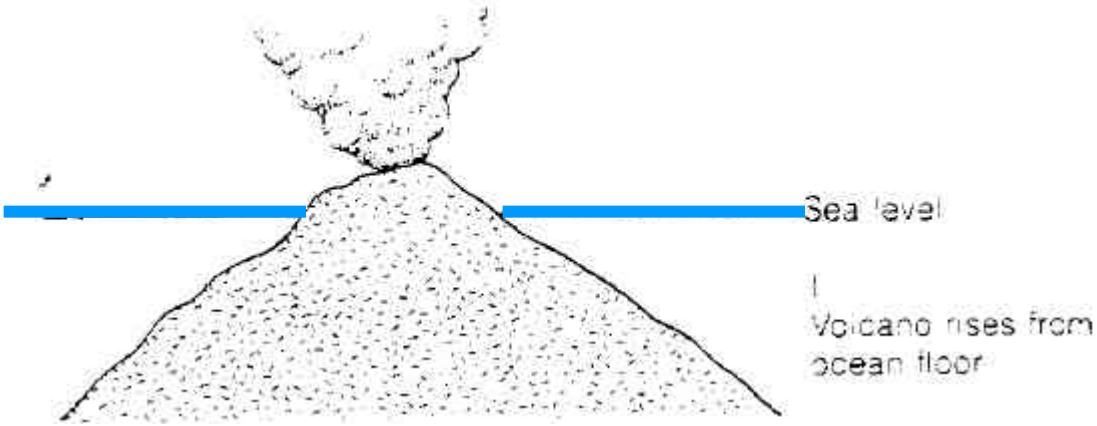
Reef Limestone … **The code base**

Figure 11-8
Diagrammatic drawing of a reef. The reef tends to grow more rapidly on the windward side, where waves bring fresh nutrients from deeper, open waters to the organisms.

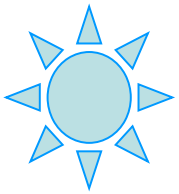
Source; Siever,Press: Earth, 1995

Stage 1: Technology breakthrough

Side-View



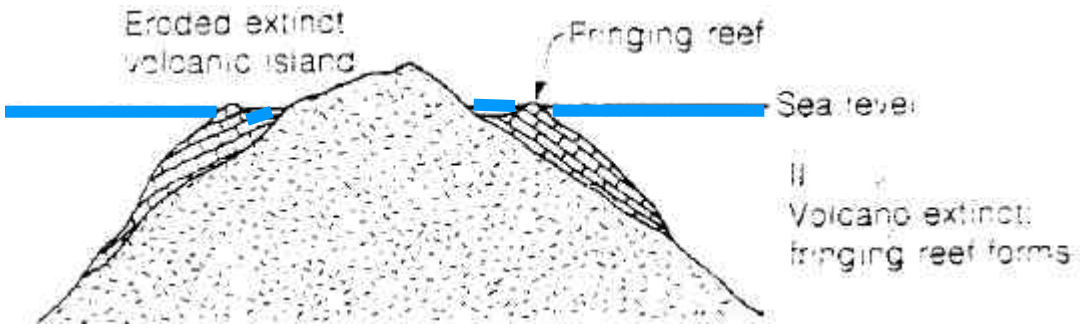
Top-View



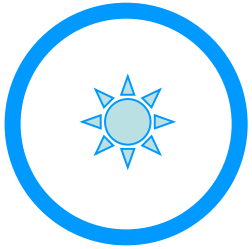
- „The volcano erupts !“
- Computation power becomes affordable
- US Army invests in the development
- First developer community

Stage 2: Community growth

Side-View



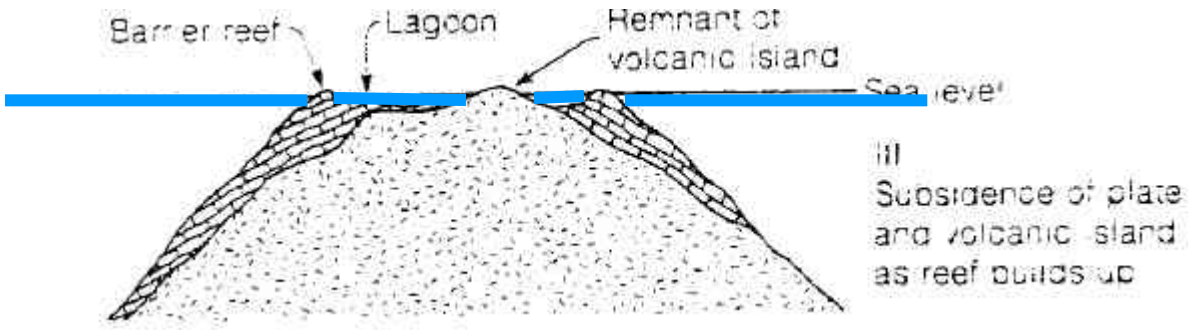
Top-View



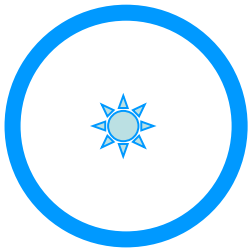
- CERL stops active development
- Code released as <u>public domain</u>
- Universities take over (Waco, Hannover)
- New developer community forms.

Stage 3: Towards a Licence

Side-View



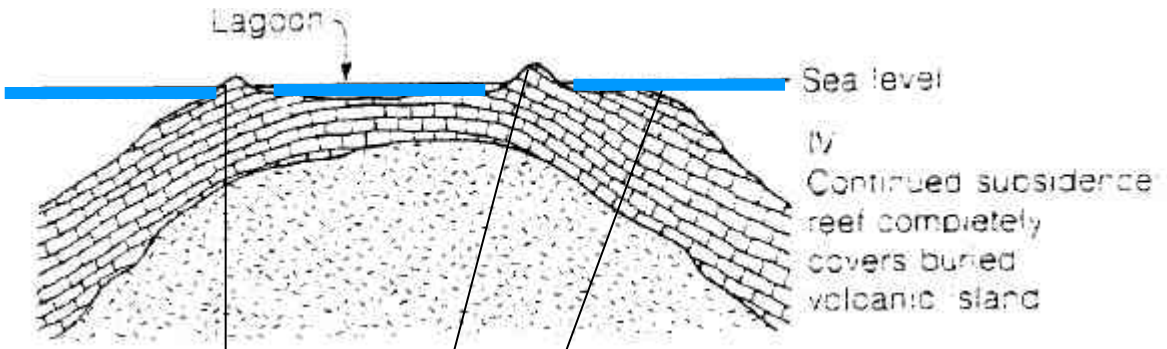
Top-View



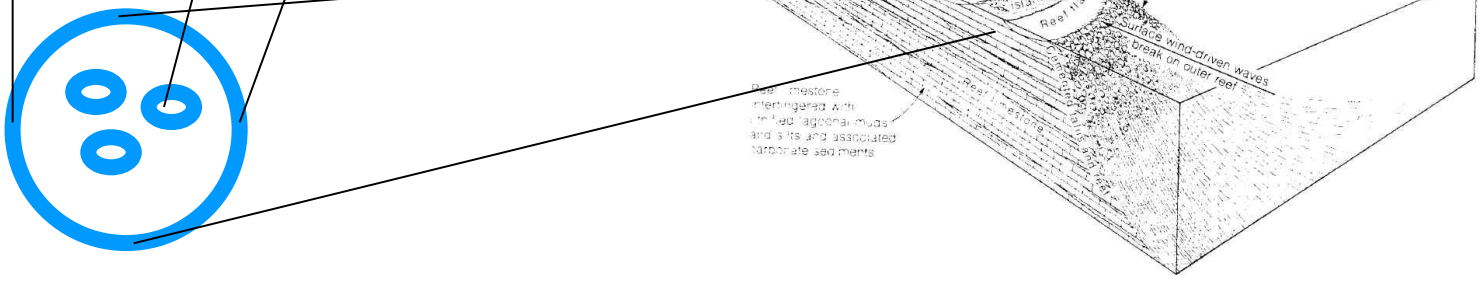
- Developer community updates and replaces CERL code base.
- 1999: General Public Licence (GPL)

Stage 4: Now

Side-View



Top-View



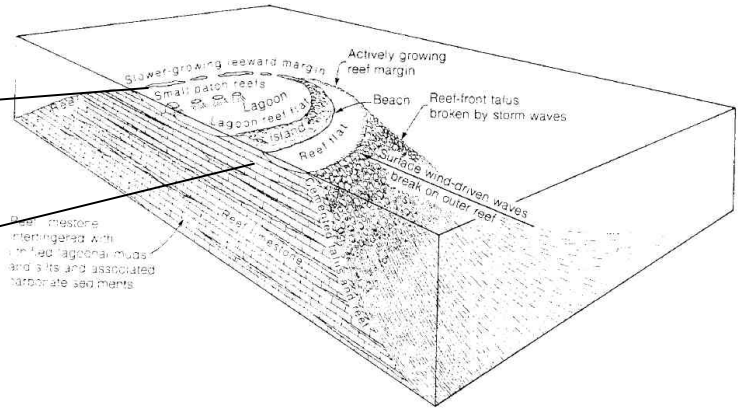


Figure 11-8
Diagrammatic drawing of a reef. The reef tends to grow more rapidly on the windward side, where waves bring fresh nutrients from deeper, open waters to the organisms.

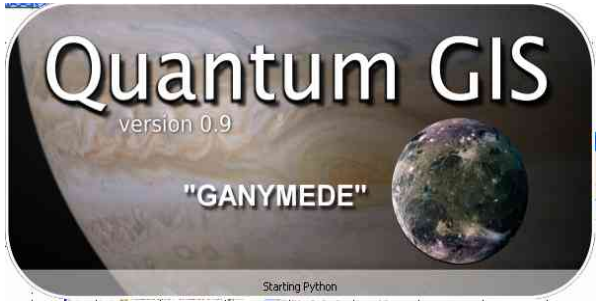
- 1999 - 2008
- GRASS is certified as FOSS by OSGEO
- Full acknowledged project beyond the incubation phase

Reef Fauna



Quantum GIS is a FOSS GIS based on Python and QT.

It provides GRASS functionality via a plug-in.



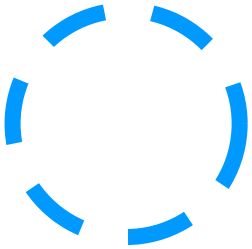
JGRASS complements the uDig Project (java-based, vector-centric) with raster-processing capabilities based on GRASS GIS.



Possible Fate: Decline

If the reef organisms (developers) can‘t keep up with the rising water level (user demands, state of the art), the reef will submerge and the project will fossilize.

Top-View



Side-View

El Capitain (Permian Reef), Texas Source: Wikipedia

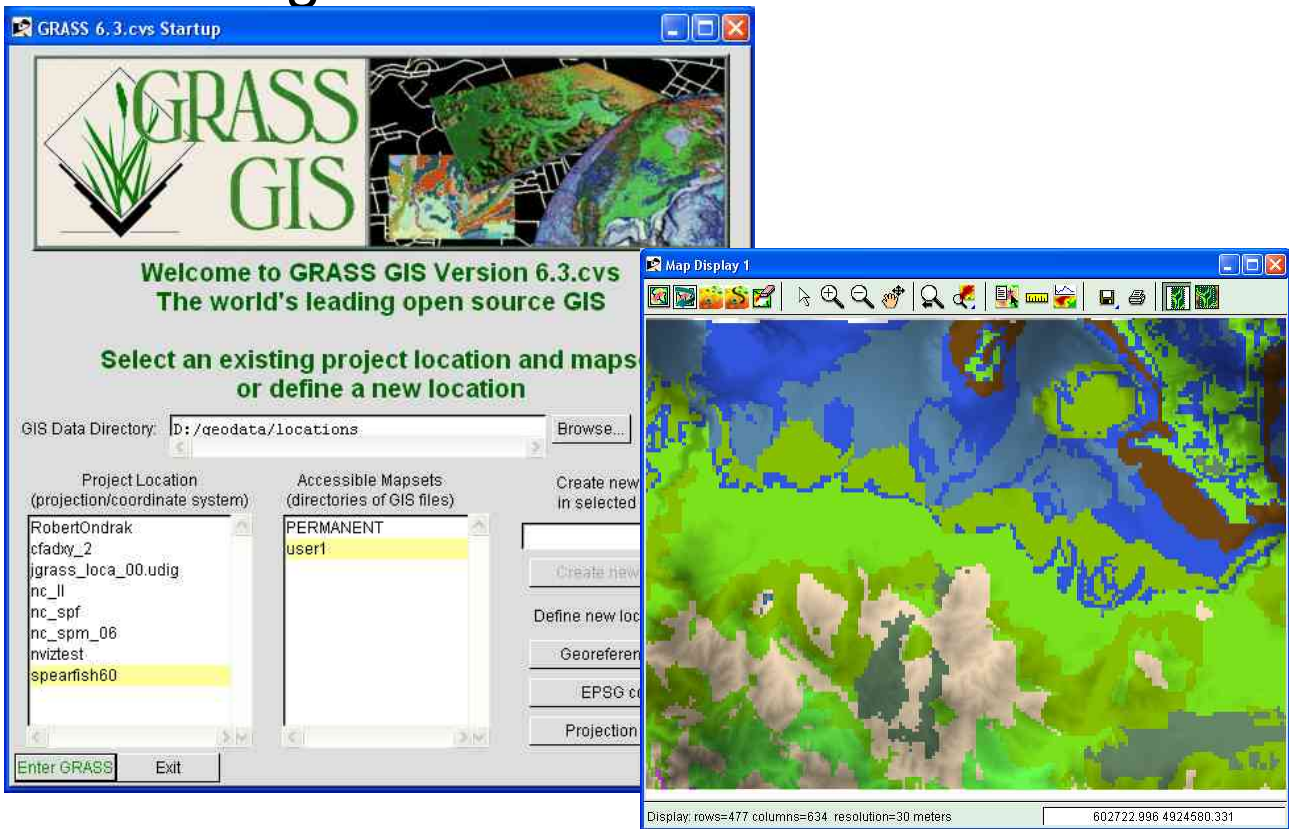


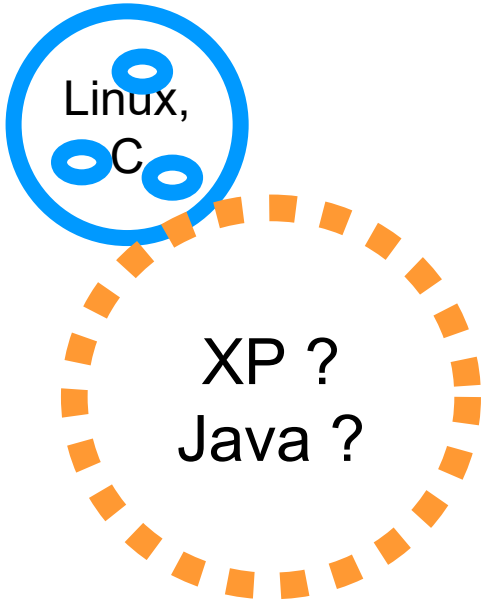
Another option: Diversification

„The reef starts to extend outside its primary niche.“

In 2007 a native GRASS version for WindowsXP became late alpha.

Challenge: will the corals thrive in the new environment…?





Coding / Recoding

- Serial Code Development: A small group of constant contributors volunteerly adds new features and updates old ones (300 modules + 80+ add-on scripts).

- The majority of the developer community is highly fluent in the code structure and uses text based tools for development such as vi or Emacs. For newcomers, this situation results in a steep learning curve. Besides, the constant overall development of libraries forces contributors of add-on C-modules to adapt their code.

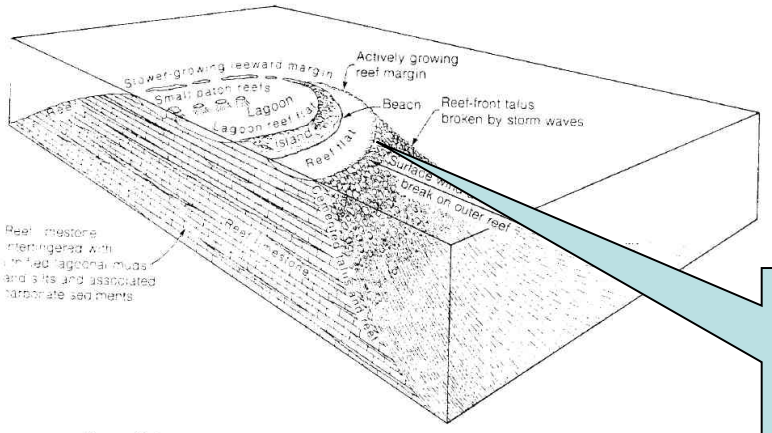
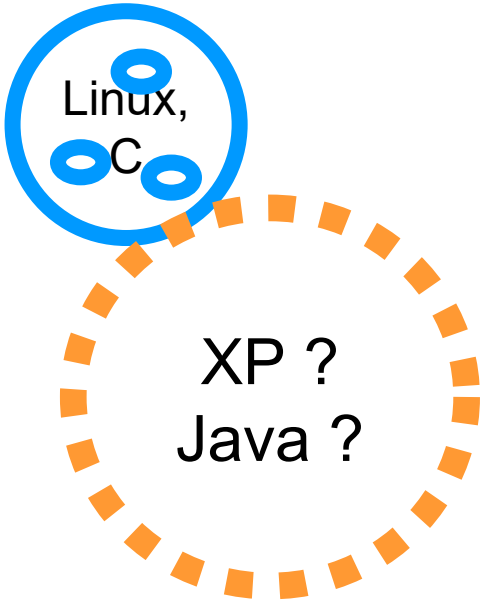


Figure 11-8
Diagrammatic drawing of a reef. The reef tends to grow more rapidly on the windward side, where waves bring fresh nutrients from deeper, open waters to the organisms.

Reef-front talus, broken by storm waves:
Abandoned code



Legacy code vs. legacy build-tools



(http://www-128.ibm.com/developerworks/aix/library/au-unix-eclipse/ Visited June 9th 2008)

- Until now, GRASS development tools were command-line based (**vi, make**).
- Integrated Development Environments (**IDE**) gives the advantages of a graphical interface, editors that support syntax highlighting, code completion, and a common platform for development.
- Using the Ant build system to adapt an existing makefile or configure script to life in an IDE.

Arguing the case for extended DIE usage

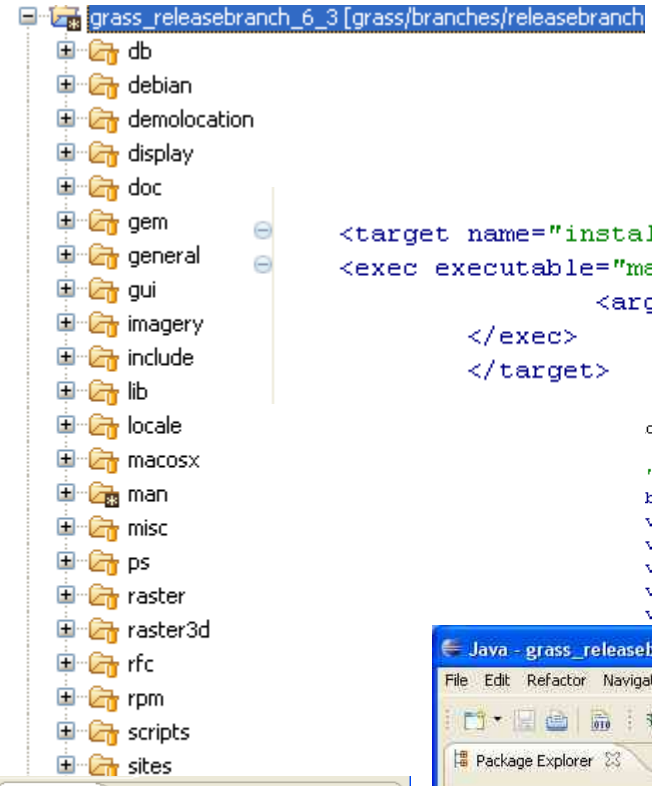
- During the last years Integrated Development Environments (IDEs) became widely available for code development.
- Many younger programmers consider the availability of IDEs as given. Therefore it makes sense to foster the know-how regarding GRASS-development with IDEs, such as Eclipse/CDT

- The decision of the GRASS GIS community to stick to C-code in the spirit of positive-legacy allowed for successful deployment of a native version for Microsoft Windows.

- Since IDEs such as Eclipse are platform independent, this also allows for active code development on non-Linux systems, such as Microsoft Windows™.

- In regard to the “dark side of legacy” such as retiring of the current pre-IDE developers and the resulting loss in know how and skill, it is even more important to enable IDE-based development to help document the code & build-process while there is time.

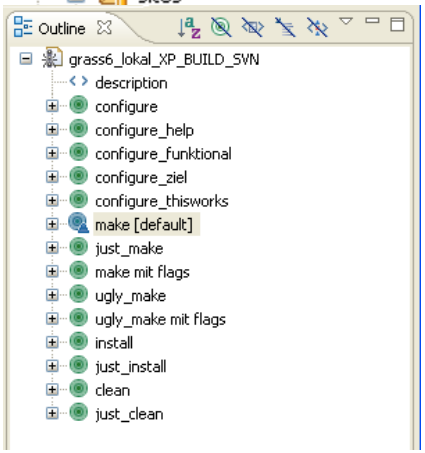
IDE-Examples

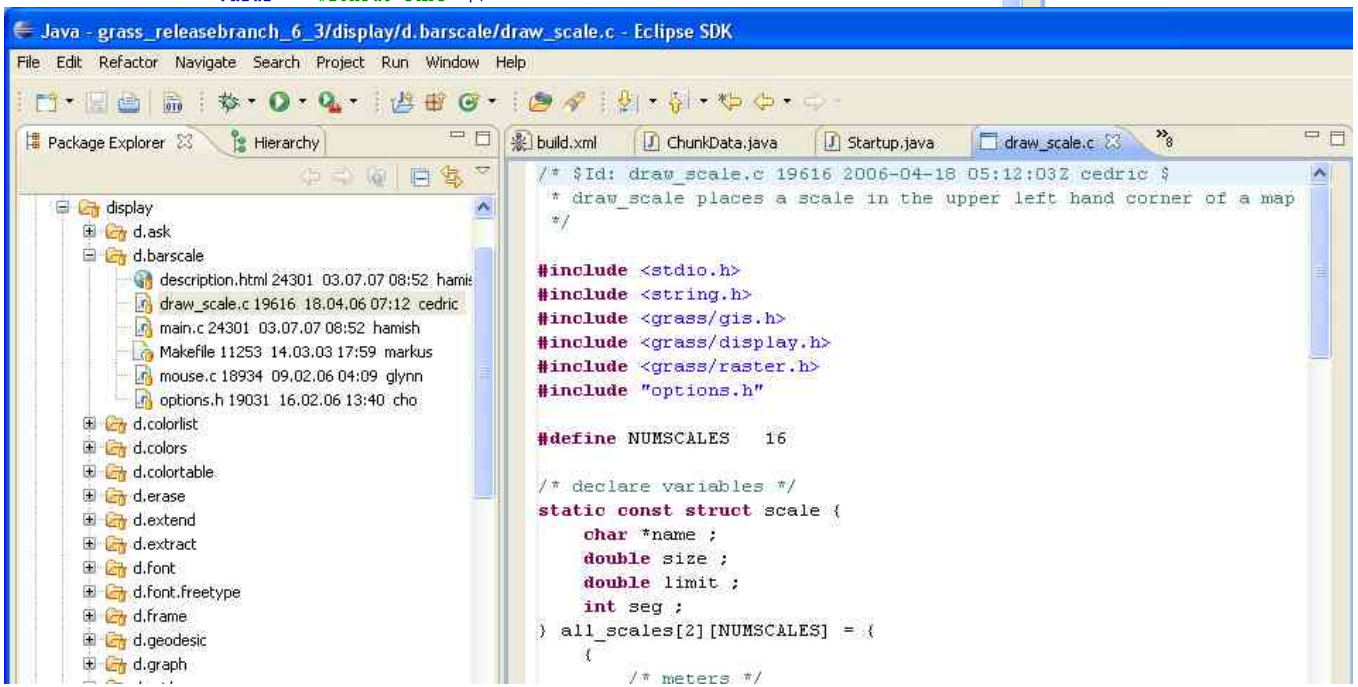



```
<target name="install" description="Install GRASS6" depends="make">
<exec executable="make">
                <arg value="install"/>
        </exec>
        </target>
```

```
                dapter fuer GRASS make

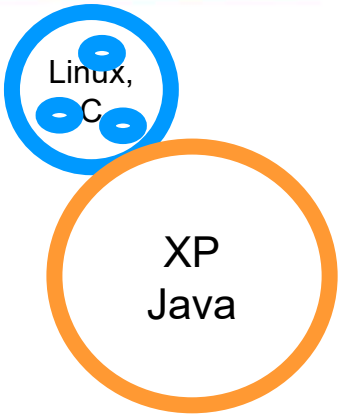
                "configure" description="Konfiguriere GRASS6xp">
                ble="C:\msys\1.0\eclipse_grass63.bat">
                value="D:\Nutzer\ploewe\workspace\grass_releasebranch_6_3\configure" />
                value="--program-suffix=eclipse_xp" />
                value="--without-x" />
                value="--without-postgres" />
                value="--without-odbc" />
```



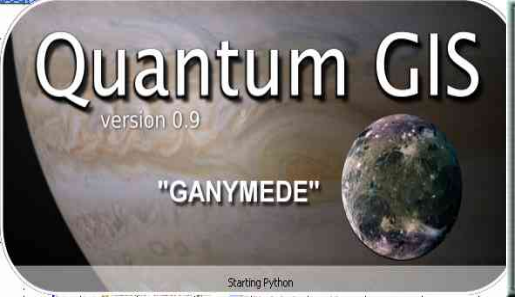


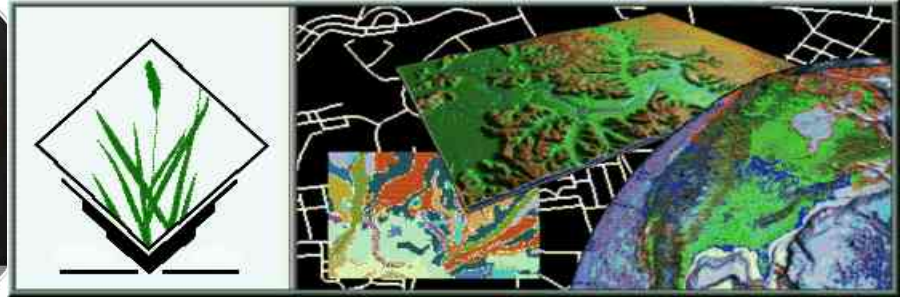
Conclusion: Taking pride in legacy





- Code Legacy is a valuable asset for community-driven software development with limited resources.
- While ensuring code functionality by the careful maintenance of existing code, the approach eventually incorporates “cutting edge“ technologies once they themselves have stood the test of time (like IDE).
- The usage of IDEs is expected to extend and rejunevate the developer community by offering alternatives to “tricky” build processes.
- It will speed up the development process and enables the active development on non-linux stystems.
- The positive, stabilizing effects of trusted legacy software which “just works well” and can be continued to be developed on various platforms.







Have a nice day

